Task 0 Execution

```
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
0
Current size of chain: 1
Difficulty of most recent block: 2
Total difficulty for all blocks: 2
Experimented with 1 hashes
Approximate hashes per second on this machine: 2195389
Expected total hashes required for the whole chain: 256.0
Nonce for most recent block: 189
```

```
Expected total hashes required for the whole chain: 256.0
Nonce for most recent block: 189
Chain hash: 00d75b7a0eca7cc48cbdd12cd89a0176474e6b4be02b1d8d09d0d9147c437eff
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
1
Enter difficulty > 1
4
Enter a transaction
Alice pays Bob 100 DSCoin
Total execution time to add this block was 247 milliseconds
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
```

```
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
1
Enter difficulty > 1
4
Enter a transaction
Bob pays Carol 20 DSCoin
Total execution time to add this block was 100 milliseconds
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
```

```
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
1
Enter difficulty > 1
4
Enter a transaction
Carol pays Donna 10 DSCoin
Total execution time to add this block was 21 milliseconds
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
3
View the Blockchain
{"ds_chain" : [{"index":0,"timestamp":"Mar 16, 2025, 8:48:49 PM","data":"Genesis","difficulty":2,"nonce":189,"previousHash":""}
```

```
View the Blockchain
{"ds_chain" : [{"index":0,"timestamp":"Mar 16, 2025, 8:48:49 PM","data":"Genesis","difficulty":2,"nonce":189,"previousHash":""}
{"index":1,"timestamp":"Mar 16, 2025, 8:50:49 PM","data":"Alice pays Bob 100 DSCoin","difficulty":4,"nonce":20992,
 "previousHash":"00d75b7a0eca7cc48cbdd12cd89a0176474e6b4be02b1d8d09d0d9147c437eff"}
{"index":2,"timestamp":"Mar 16, 2025, 8:51:13 PM","data":"Bob pays Carol 20 DSCoin","difficulty":4,"nonce":59328,
 "previousHash":"0000f1cfe2c651f9413abdfe47cef6025d363d56e7a045c443bce6f2afcc90d5"}
{"index":3,"timestamp":"Mar 16, 2025, 8:51:24 PM","data":"Carol pays Donna 10 DSCoin","difficulty":4,"nonce":13737,
 "previousHash":"00007294d16f8ebcf9d5d69934d24724fd667bb685809d213d2c26e4d9be40dd"}
], "chainHash":"00007a126f4e378dc1efb4b152611f4b40d7e1221c65ec4a1d9a800e7cfea44c"}
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
2
True
Total execution time required to verify the chain was 1 milliseconds
```

```
True
Total execution time required to verify the chain was 1 milliseconds
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
4
Corrupt the chain
Enter block ID of block to corrupt
2
Enter new data for block 2
Bob pays Tony 30 DSCoin
Block 2 now holds Bob pays Tony 30 DSCoin
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
```

```
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
3
View the Blockchain
{"ds_chain" : [{"index":0,"timestamp":"Mar 16, 2025, 8:48:49 PM","data":"Genesis","difficulty":2,"nonce":189,"previousHash":""}
{"index":1,"timestamp":"Mar 16, 2025, 8:50:49 PM","data":"Alice pays Bob 100 DSCoin","difficulty":4,"nonce":20992,
 "previousHash":"00d75b7a0eca7cc48cbdd12cd89a0176474e6b4be02b1d8d09d0d9147c437eff"}
{"index":2,"timestamp":"Mar 16, 2025, 8:51:13 PM","data":"Bob pays Tony 30 DSCoin","difficulty":4,"nonce":59328,
 "previousHash":"0000f1cfe2c651f9413abdfe47cef6025d363d56e7a045c443bce6f2afcc90d5"}
{"index":3,"timestamp":"Mar 16, 2025, 8:51:24 PM","data":"Carol pays Donna 10 DSCoin","difficulty":4,"nonce":13737,
 "previousHash":"00007294d16f8ebcf9d5d69934d24724fd667bb685809d213d2c26e4d9be40dd"}
], "chainHash":"00007a126f4e378dc1efb4b152611f4b40d7e1221c65ec4a1d9a800e7cfea44c"}
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
```

```
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
2
Chain Verification: FALSE
Improper hash on node 1 Does not begin with 0000
Total execution time required to verify the chain was 0 milliseconds
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
5
```

```
Enter choice:
5
Repairing the entire chain
Total execution time required to repair the chain was 574 milliseconds
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
2
True
Total execution time required to verify the chain was 0 milliseconds
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
```

```
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
3
View the Blockchain
{"ds_chain" : [{"index":0,"timestamp":"Mar 16, 2025, 8:48:49 PM","data":"Genesis","difficulty":2,"nonce":189,"previousHash":""}
{"index":1,"timestamp":"Mar 16, 2025, 8:50:49 PM","data":"Alice pays Bob 100 DSCoin","difficulty":4,"nonce":20992,
 "previousHash":"00d75b7a0eca7cc48cbdd12cd89a0176474e6b4be02b1d8d09d0d9147c437eff"}
{"index":2,"timestamp":"Mar 16, 2025, 8:51:13 PM","data":"Bob pays Tony 30 DSCoin","difficulty":4,"nonce":234523,
 "previousHash":"0000f1cfe2c651f9413abdfe47cef6025d363d56e7a045c443bce6f2afcc90d5"}
{"index":3,"timestamp":"Mar 16, 2025, 8:51:24 PM","data":"Carol pays Donna 10 DSCoin","difficulty":4,"nonce":262687,
 "previousHash":"0000258b6bf8534ff6a81c94913bfec1338aab6b24454e692ddd2a9ace44c6bd"}
], "chainHash":"0000590eb0dd40db20a147b30eda04b82ec7c2e112e88fa862dd730082634983"}
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
```

```
{"ds_chain" : [{"index":0,"timestamp":"Mar 16, 2025, 8:48:49 PM","data":"Genesis","difficulty":2,"nonce":189,"previousHash":""}
{"index":1,"timestamp":"Mar 16, 2025, 8:50:49 PM","data":"Alice pays Bob 100 DSCoin","difficulty":4,"nonce":20992,
 "previousHash":"00d75b7a0eca7cc48cbdd12cd89a0176474e6b4be02b1d8d09d0d9147c437eff"}
{"index":2,"timestamp":"Mar 16, 2025, 8:51:13 PM","data":"Bob pays Tony 30 DSCoin","difficulty":4,"nonce":234523,
 "previousHash":"0000f1cfe2c651f9413abdfe47cef6025d363d56e7a045c443bce6f2afcc90d5"}
{"index":3,"timestamp":"Mar 16, 2025, 8:51:24 PM","data":"Carol pays Donna 10 DSCoin","difficulty":4,"nonce":262687,
 "previousHash":"0000258b6bf8534ff6a81c94913bfec1338aab6b24454e692ddd2a9ace44c6bd"}
], "chainHash":"0000590eb0dd40db20a147b30eda04b82ec7c2e112e88fa862dd730082634983"}
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
6

Process finished with exit code 0
```


Task 0 Block.java

```
import java.math.BigInteger;
import java.security.MessageDigest;

import com.google.gson.Gson;

public class Block {

    private int index;
    private java.sql.Timestamp timestamp;
    private java.lang.String data;
    private int difficulty;
    private java.math.BigInteger nonce = BigInteger.ZERO;
    private java.lang.String previousHash = "";

    public Block(int index, java.sql.Timestamp timestamp, java.lang.String data, int difficulty) {
        this.index = index;
        this.timestamp = timestamp;
        this.data = data;
        this.difficulty = difficulty;
    }

    public java.lang.String getData(){return data;}
    public int getDifficulty(){return difficulty;}
    public int getIndex(){return index;}
    public java.math.BigInteger getNonce(){return nonce;}
```

```
    public java.lang.String getPreviousHash(){return previousHash;}
    public java.sql.Timestamp getTimestamp(){return timestamp;}

    public void setData(java.lang.String data){this.data = data;}
    public void setDifficulty(int difficulty){this.difficulty = difficulty;}
    public void setIndex(int index){this.index = index;}
    public void setPreviousHash(java.lang.String previousHash){this.previousHash =
previousHash;}
    public void setTimestamp(java.sql.Timestamp timestamp){this.timestamp = timestamp;}

    public java.lang.String calculateHash(){
        try{
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        String input = index + timestamp.toString() + data + difficulty + previousHash + nonce;
        byte[] hashBytes = digest.digest(input.getBytes("UTF-8"));
        StringBuilder hexString = new StringBuilder();

        for (byte b : hashBytes) {
            String hex = Integer.toHexString(0xff & b);
            if (hex.length() == 1) {
                hexString.append('0');
            }
            hexString.append(hex);
        }

        return hexString.toString();
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
    }

    public java.lang.String proofOfWork(){
        String target =  String.format("%0"+difficulty+"d",0);
        while(true){
            String hash = calculateHash();
            if(hash.startsWith(target)){
                return calculateHash();
            }
```

```
            nonce = nonce.add(BigInteger.ONE);
        }
    }
    public static void main(java.lang.String[] args){}

    public java.lang.String toString(){
        Gson gson = new Gson();
        return gson.toJson(this);
    }
}
```

Task 0 BlockChain.java

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.security.MessageDigest;
import java.security.Timestamp;
import java.util.ArrayList;

public class BlockChain {
    private ArrayList<Block> blocks;
    private String chainHash;
    private int hps;
    public BlockChain(ArrayList<Block> blocks, String chainHash, int hps) {
        this.blocks = new ArrayList<Block>();
        this.chainHash = "";
        this.hps = 0;
        Block genesisBlock = new Block(0,new
java.sql.Timestamp(System.currentTimeMillis()),"Genesis",2);
        genesisBlock.setPreviousHash("");
        genesisBlock.proofOfWork();
        this.blocks.add(genesisBlock);
        this.chainHash = genesisBlock.calculateHash();
    }
```

```
public void addBlock(Block newBlock) {
   Block previousBlock = blocks.get(blocks.size()-1);
   newBlock.setPreviousHash(previousBlock.calculateHash());
   newBlock.proofOfWork();
   chainHash = newBlock.calculateHash();
   blocks.add(newBlock);
}
public void computeHashesPerSecond(){
   String hashString = "oooooooo";
   long start = System.currentTimeMillis();
   try{
      MessageDigest md = MessageDigest.getInstance("SHA-256");
      for( int i =0; i<=2000000; i++){
         md.update(hashString.getBytes());
         md.digest();
      }
   }catch(Exception e){
      e.printStackTrace();
   }
   long end = System.currentTimeMillis();
   double  duration = (end-start)/1000.0;
   hps = (int)(2000000/duration);
}
public void repairChain(){
   for(int i = 0; i<blocks.size(); i++){
      Block currentBlock = blocks.get(i);
      currentBlock.proofOfWork();
      if(i<blocks.size()-1){
         Block nextBlock = blocks.get(i+1);
         nextBlock.setPreviousHash(currentBlock.calculateHash());
      }
   }
   this.chainHash = blocks.get(blocks.size()-1).calculateHash();
}
public Block getBlock(int i){return blocks.get(i);}
public Block getLatestBlock(){return blocks.get(blocks.size()-1);}
public java.lang.String getChainHash(){return this.chainHash;}
public int getChainSize(){return blocks.size();}
```

```
    public int getHashesPerSecond(){return this.hps;}
    public java.sql.Timestamp getTime(){return new
java.sql.Timestamp(System.currentTimeMillis());}
    public int getTotalDifficulty(){
        int difficulty = 0;
        for(Block block: blocks){
            difficulty += block.getDifficulty();
        }
        return  difficulty;
    }
    public double getExpectedHashes(){
        double total = 0.0;
        for(Block block: blocks){
            total+= Math.pow(16,block.getDifficulty());
        }
        return total;
    }
    public java.lang.String isChainValid() {
        if(blocks.size()==1){
            Block genesisBlock = blocks.get(0);
            String genesisHash = genesisBlock.calculateHash();
            String requiredPrefix = "0".repeat(genesisBlock.getDifficulty()); // Corrected prefix
generation
            if (!genesisHash.startsWith(requiredPrefix)) {
                return "Chain Verification: FALSE\nGenesis block's proof of work is invalid";
            }
            if (!genesisHash.equals(this.chainHash)) {
                return "Chain Verification: FALSE\nGenesis block's hash does not match the Chain
hash";
            }
        }

        for (int i = 1; i < blocks.size(); i++) {
            Block currentBlock = blocks.get(i);
            Block previousBlock = blocks.get(i - 1);
            if (!currentBlock.getPreviousHash().equals(previousBlock.calculateHash())) {
                return "Chain Verification: FALSE\nPrevious block's hash does not match the Chain
hash";
```

```
            }
            String currentRequiredPrefix = "0".repeat(currentBlock.getDifficulty()); // Corrected
prefix generation
            if (!currentBlock.calculateHash().startsWith(currentRequiredPrefix)) {
                return "Chain Verification: FALSE\nImproper hash on node " + (i - 1) + " Does not begin
with " + currentRequiredPrefix;
            }
        }
        Block lastBlock = blocks.get(blocks.size() - 1);
        if (!chainHash.equals(lastBlock.calculateHash())) {
            return "Chain Verification: FALSE\nChain's hash does not match the last block's hash";
        }
        return "True";
    }

    public java.lang.String toString(){
        StringBuilder str = new StringBuilder();
        str.append("{\"ds_chain\" : [");
        for(Block b : blocks){
            str.append(b.toString()).append("\n");
        }
        str.append("], \"chainHash\":\"").append(chainHash).append("\"}");
        return str.toString();
    }
    public static void main(String[] args){
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        BlockChain blockChain = new BlockChain(new ArrayList<Block>(), "",0);
        blockChain.computeHashesPerSecond();
        try{
            while(true){
                System.out.println("""
                        0. View basic blockchain status.
                        1. Add a transaction to the blockchain.
                        2. Verify the blockchain.
                        3. View the blockchain.
                        4. Corrupt the chain.
                        5. Hide the corruption by repairing the chain.
                        6. Exit""");
```

```
            System.out.println("Enter choice:");
            String choice = br.readLine();
            switch (choice){
               case "0":
                  System.out.println("Current size of chain: "+blockChain.getChainSize());
                  System.out.println("Difficulty of most recent block:
"+blockChain.getLatestBlock().getDifficulty());
                  System.out.println("Total difficulty for all blocks: "+blockChain.getTotalDifficulty());
                  System.out.println("Experimented with "+blockChain.getChainSize()+" hashes");
                  System.out.println("Approximate hashes per second on this machine:
"+blockChain.getHashesPerSecond());
                  System.out.println("Expected total hashes required for the whole chain:
"+blockChain.getExpectedHashes());
                  System.out.println("Nonce for most recent block:
"+blockChain.getLatestBlock().getNonce());
                  System.out.println("Chain hash: "+blockChain.getChainHash());
                  break;
               case "1":
                  System.out.println("Enter difficulty > 1");
                  String difficulty = br.readLine();
                  System.out.println("Enter a transaction");
                  String transaction = br.readLine();
                  Block block = new Block(blockChain.getChainSize(),new
java.sql.Timestamp(System.currentTimeMillis()),transaction,Integer.parseInt(difficulty));
                  long startTime = System.currentTimeMillis();
                  blockChain.addBlock(block);
                  long endTime = System.currentTimeMillis();
                  System.out.println("Total execution time to add this block was "+(endTime-
startTime)+" milliseconds");
                  break;
               case "2":
                  long startTime2 = System.currentTimeMillis();
                  System.out.println(blockChain.isChainValid());
                  long endTime2 = System.currentTimeMillis();
                  System.out.println("Total execution time required to verify the chain was
"+(endTime2-startTime2)+" milliseconds");
                  break;
               case "3":
```

```
                System.out.println("View the Blockchain");
                System.out.println(blockChain.toString());
                break;
            case "4":
                System.out.println("Corrupt the chain");
                System.out.println("Enter block ID of block to corrupt");
                int id = Integer.parseInt(br.readLine());
                System.out.println("Enter new data for block "+id);
                String data = br.readLine();
                blockChain.getBlock(id).setData(data);
                System.out.println("Block "+id+" now holds "+data);
                break;
            case "5":
                System.out.println("Repairing the entire chain");
                long startTime3 = System.currentTimeMillis();
                blockChain.repairChain();
                long endTime3 = System.currentTimeMillis();
                System.out.println("Total execution time required to repair the chain was
"+(endTime3-startTime3)+" milliseconds");
                break;
            case "6":
                return;
            default:
                System.out.println("Invalid choice");
                break;
          }
        }
      }catch(Exception e){
        e.printStackTrace();
      }
    }
}
```

Task 1 Client Side Execution

```
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
0
Current size of chain: 1
Difficulty of most recent block: 2
Total difficulty for all blocks: 2
Experimented with 1 hashes
Approximate hashes per second on this machine: 4073319
Expected total hashes required for the whole chain: 256.0
Nonce for most recent block: 0
Chain hash: 00263e9afb48768ba6a36c519bd597a34719ecef2b328ff1e90d88a70e91cfd5
0. View basic blockchain status.
```

```
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
1
Enter difficulty > 1
4
Enter a transaction
Alice pays Bob 100 DSCoin
Total execution time to add this block was 485 milliseconds
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
```

```
Enter choice:
1
Enter difficulty > 1
4
Enter a transaction
Bob pays Carol 20 DSCoin
Total execution time to add this block was 102 milliseconds
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
1
Enter difficulty > 1
4
Enter a transaction
Carol pays Donna 10 DSCoin
Total execution time to add this block was 100 milliseconds
0. View basic blockchain status.
```

```
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
3
View the Blockchain
{"ds_chain" : [{"index":0,"timestamp":"Mar 16, 2025, 8:19:38 PM","data":"Genesis","difficulty":2,"nonce":0,"previousHash":""}
{"index":1,"timestamp":"Mar 16, 2025, 8:20:30 PM","data":"Alice pays Bob 100 DSCoin","difficulty":4,"nonce":191044,
 "previousHash":"00263e9afb48768ba6a36c519bd597a34719ecef2b328ff1e90d88a70e91cfd5"}
{"index":2,"timestamp":"Mar 16, 2025, 8:20:47 PM","data":"Bob pays Carol 20 DSCoin","difficulty":4,"nonce":27371,
 "previousHash":"00000ce74253cdc42889c5f0c8c95f28ac21f7e5a7e47f6d2c0ee9bdebbb4721"}
{"index":3,"timestamp":"Mar 16, 2025, 8:21:00 PM","data":"Carol pays Donna 10 DSCoin","difficulty":4,"nonce":84905,
 "previousHash":"000015a08112b18f8789da8bed1fbe38bc9d1238d377d94b0e7134f7fd432b96"}
], "chainHash":"0000523bcb1d33e740bb6412aa324d4ef11e184a234b435b715617a750d34885"}
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
```

```
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
2
Verifying the entire chain
Chain verification: True
Total execution time required to verify the chain was 0 milliseconds
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
4
Corrupt the chain
Enter block ID of block to corrupt
2
Enter new data for block 2
```

```
Enter new data for block 2
Bob pays Tony 30 DSCoin
Block 2 now holds Bob pays Tony 30 DSCoin
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
3
View the Blockchain
{"ds_chain" : [{"index":0,"timestamp":"Mar 16, 2025, 8:19:38 PM","data":"Genesis","difficulty":2,"nonce":0,"previousHash":""}
{"index":1,"timestamp":"Mar 16, 2025, 8:20:30 PM","data":"Alice pays Bob 100 DSCoin","difficulty":4,"nonce":191044,
 "previousHash":"00263e9afb48768ba6a36c519bd597a34719ecef2b328ff1e90d88a70e91cfd5"}
{"index":2,"timestamp":"Mar 16, 2025, 8:20:47 PM","data":"Bob pays Tony 30 DSCoin","difficulty":4,"nonce":27371,
 "previousHash":"00000ce74253cdc42889c5f0c8c95f28ac21f7e5a7e47f6d2c0ee9bdebbb4721"}
{"index":3,"timestamp":"Mar 16, 2025, 8:21:00 PM","data":"Carol pays Donna 10 DSCoin","difficulty":4,"nonce":84905,
 "previousHash":"000015a08112b18f8789da8bed1fbe38bc9d1238d377d94b0e7134f7fd432b96"}
], "chainHash":"0000523bcb1d33e740bb6412aa324d4ef11e184a234b435b715617a750d34885"}
0. View basic blockchain status.
```

```
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
2
Verifying the entire chain
Chain verification: Chain Verification: FALSE
Improper hash on node 1 Does not begin with 0000
Total execution time required to verify the chain was 0 milliseconds
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
5
```

```
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
5
Repairing the entire chain
Total execution time required to repair the chain was 54 milliseconds
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
2
Verifying the entire chain
Chain verification: True
Total execution time required to verify the chain was 0 milliseconds
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
```

```
6. Exit
Enter choice:
3
View the Blockchain
{"ds_chain" : [{"index":0,"timestamp":"Mar 16, 2025, 8:19:38 PM","data":"Genesis","difficulty":2,"nonce":0,"previousHash":""}
{"index":1,"timestamp":"Mar 16, 2025, 8:20:30 PM","data":"Alice pays Bob 100 DSCoin","difficulty":4,"nonce":191044,
 "previousHash":"00263e9afb48768ba6a36c519bd597a34719ecef2b328ff1e90d88a70e91cfd5"}
{"index":2,"timestamp":"Mar 16, 2025, 8:20:47 PM","data":"Bob pays Tony 30 DSCoin","difficulty":4,"nonce":36236,
 "previousHash":"00000ce74253cdc42889c5f0c8c95f28ac21f7e5a7e47f6d2c0ee9bdebbb4721"}
{"index":3,"timestamp":"Mar 16, 2025, 8:21:00 PM","data":"Carol pays Donna 10 DSCoin","difficulty":4,"nonce":133395,
 "previousHash":"00009e8a83c2fe71079659022cc35955668fec6943b26070a838c0c3189d595a"}
], "chainHash":"00009dce700b680da170414278b66f57b2a374377bec904c6b771a3733196ed5"}
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
6
```

```
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
6

Process finished with exit code 0
```

Task 1 Server Side Execution

```
Server is running...
Waiting for a client...
New client connected.
We have a visitor
THE JSON REQUEST MESSAGE IS SHOWN HERE
{"choice":"0","command":""}
THE JSON RESPONSE MESSAGE IS SHOWN HERE
{"message":"Current size of chain: 1\nDifficulty of most recent block: 2\nTotal difficulty for all blocks: 2\nExperimented with 1
 hashes\nApproximate hashes per second on this machine: 4073319\nExpected total hashes required for the whole chain: 256.0\nNonce for
 most recent block: 0\nChain hash: 00263e9afb48768ba6a36c519bd597a34719ecef2b328ff1e90d88a70e91cfd5"}
Number of Blocks on Chain == 1
We have a visitor
THE JSON REQUEST MESSAGE IS SHOWN HERE
{"choice":"1","command":"4,Alice pays Bob 100 DSCoin"}
THE JSON RESPONSE MESSAGE IS SHOWN HERE
{"message":"Total execution time to add this block was 485 milliseconds"}
Number of Blocks on Chain == 2
We have a visitor
```

```
We have a visitor
THE JSON REQUEST MESSAGE IS SHOWN HERE
{"choice":"1","command":"4,Bob pays Carol 20 DSCoin"}
THE JSON RESPONSE MESSAGE IS SHOWN HERE
{"message":"Total execution time to add this block was 102 milliseconds"}
Number of Blocks on Chain == 3
We have a visitor
THE JSON REQUEST MESSAGE IS SHOWN HERE
{"choice":"1","command":"4,Carol pays Donna 10 DSCoin"}
THE JSON RESPONSE MESSAGE IS SHOWN HERE
{"message":"Total execution time to add this block was 100 milliseconds"}
Number of Blocks on Chain == 4
We have a visitor
THE JSON REQUEST MESSAGE IS SHOWN HERE
{"choice":"3","command":""}
THE JSON RESPONSE MESSAGE IS SHOWN HERE
{"message":"View the Blockchain\n{\"ds_chain\" : [{\"index\":0,\"timestamp\":\"Mar 16, 2025, 8:19:38 PM\",\"data\":\"Genesis\",
 \"difficulty\":2,\"nonce\":0,\"previousHash\":\"\"}\n{\"index\":1,\"timestamp\":\"Mar 16, 2025, 8:20:30 PM\",\"data\":\"Alice pays Bob
 100 DSCoin\",\"difficulty\":4,\"nonce\":191044,\"previousHash\":\"00263e9afb48768ba6a36c519bd597a34719ecef2b328ff1e90d88a70e91cfd5\"}\n
 {\"index\":2,\"timestamp\":\"Mar 16, 2025, 8:20:47 PM\",\"data\":\"Bob pays Carol 20 DSCoin\",\"difficulty\":4,\"nonce\":27371,
 \"previousHash\":\"00000ce74253cdc42889c5f0c8c95f28ac21f7e5a7e47f6d2c0ee9bdebbb4721\"}\n{\"index\":3,\"timestamp\":\"Mar 16, 2025,
 8:21:00 PM\",\"data\":\"Carol pays Donna 10 DSCoin\",\"difficulty\":4,\"nonce\":84905,
```

```
 8:21:00 PM\",\"data\":\"Carol pays Donna 10 DSCoin\",\"difficulty\":4,\"nonce\":84905,
 \"previousHash\":\"000015a08112b18f8789da8bed1fbe38bc9d1238d377d94b0e7134f7fd432b96\"}\n],
 \"chainHash\":\"0000523bcb1d33e740bb6412aa324d4ef11e184a234b435b715617a750d34885\"}"}
Number of Blocks on Chain == 4
We have a visitor
THE JSON REQUEST MESSAGE IS SHOWN HERE
{"choice":"2","command":""}
THE JSON RESPONSE MESSAGE IS SHOWN HERE
{"message":"Verifying the entire chain\nChain verification: True\nTotal execution time required to verify the chain was 0 milliseconds"}
Number of Blocks on Chain == 4
We have a visitor
THE JSON REQUEST MESSAGE IS SHOWN HERE
{"choice":"4","command":"2,Bob pays Tony 30 DSCoin"}
THE JSON RESPONSE MESSAGE IS SHOWN HERE
{"message":"Block 2 now holds Bob pays Tony 30 DSCoin"}
Number of Blocks on Chain == 4
We have a visitor
THE JSON REQUEST MESSAGE IS SHOWN HERE
{"choice":"3","command":""}
THE JSON RESPONSE MESSAGE IS SHOWN HERE
{"message":"View the Blockchain\n{\"ds_chain\" : [{\"index\":0,\"timestamp\":\"Mar 16, 2025, 8:19:38 PM\",\"data\":\"Genesis\",
```

```
{"message":"View the Blockchain\n{\"ds_chain\" : [{\"index\":0,\"timestamp\":\"Mar 16, 2025, 8:19:38 PM\",\"data\":\"Genesis\",
 \"difficulty\":2,\"nonce\":0,\"previousHash\":\"\"}\n{\"index\":1,\"timestamp\":\"Mar 16, 2025, 8:20:30 PM\",\"data\":\"Alice pays Bob
 100 DSCoin\",\"difficulty\":4,\"nonce\":191044,\"previousHash\":\"00263e9afb48768ba6a36c519bd597a34719ecef2b328ff1e90d88a70e91cfd5\"}\n
 {\"index\":2,\"timestamp\":\"Mar 16, 2025, 8:20:47 PM\",\"data\":\"Bob pays Tony 30 DSCoin\",\"difficulty\":4,\"nonce\":27371,
 \"previousHash\":\"00000ce74253cdc42889c5f0c8c95f28ac21f7e5a7e47f6d2c0ee9bdebbb4721\"}\n{\"index\":3,\"timestamp\":\"Mar 16, 2025,
 8:21:00 PM\",\"data\":\"Carol pays Donna 10 DSCoin\",\"difficulty\":4,\"nonce\":84905,
 \"previousHash\":\"000015a08112b18f8789da8bed1fbe38bc9d1238d377d94b0e7134f7fd432b96\"}\n],
 \"chainHash\":\"0000523bcb1d33e740bb6412aa324d4ef11e184a234b435b715617a750d34885\"}"}
Number of Blocks on Chain == 4
We have a visitor
THE JSON REQUEST MESSAGE IS SHOWN HERE
{"choice":"2","command":""}
THE JSON RESPONSE MESSAGE IS SHOWN HERE
{"message":"Verifying the entire chain\nChain verification: Chain Verification: FALSE\nImproper hash on node 1 Does not begin with
 0000\nTotal execution time required to verify the chain was 0 milliseconds"}
Number of Blocks on Chain == 4
We have a visitor
THE JSON REQUEST MESSAGE IS SHOWN HERE
{"choice":"5","command":""}
THE JSON RESPONSE MESSAGE IS SHOWN HERE
```

```
THE JSON RESPONSE MESSAGE IS SHOWN HERE
{"message":"Repairing the entire chain\nTotal execution time required to repair the chain was 54 milliseconds"}
Number of Blocks on Chain == 4
We have a visitor
THE JSON REQUEST MESSAGE IS SHOWN HERE
{"choice":"2","command":""}
THE JSON RESPONSE MESSAGE IS SHOWN HERE
{"message":"Verifying the entire chain\nChain verification: True\nTotal execution time required to verify the chain was 0 milliseconds"}
Number of Blocks on Chain == 4
We have a visitor
THE JSON REQUEST MESSAGE IS SHOWN HERE
{"choice":"3","command":""}
THE JSON RESPONSE MESSAGE IS SHOWN HERE
{"message":"View the Blockchain\n{\"ds_chain\" : [{\"index\":0,\"timestamp\":\"Mar 16, 2025, 8:19:38 PM\",\"data\":\"Genesis\",
 \"difficulty\":2,\"nonce\":0,\"previousHash\":\"\"}\n{\"index\":1,\"timestamp\":\"Mar 16, 2025, 8:20:30 PM\",\"data\":\"Alice pays Bob
 100 DSCoin\",\"difficulty\":4,\"nonce\":191044,\"previousHash\":\"00263e9afb48768ba6a36c519bd597a34719ecef2b328ff1e90d88a70e91cfd5\"}\n
 {\"index\":2,\"timestamp\":\"Mar 16, 2025, 8:20:47 PM\",\"data\":\"Bob pays Tony 30 DSCoin\",\"difficulty\":4,\"nonce\":36236,
 \"previousHash\":\"00000ce74253cdc42889c5f0c8c95f28ac21f7e5a7e47f6d2c0ee9bdebbb4721\"}\n{\"index\":3,\"timestamp\":\"Mar 16, 2025,
 8:21:00 PM\",\"data\":\"Carol pays Donna 10 DSCoin\",\"difficulty\":4,\"nonce\":133395,
 \"previousHash\":\"00009e8a83c2fe71079659022cc35955668fec6943b26070a838c0c3189d595a\"}\n],
 \"chainHash\":\"00009dce700b680da170414278b66f57b2a374377bec904c6b771a3733196ed5\"}"}
Number of Blocks on Chain == 4
```

```
THE JSON RESPONSE MESSAGE IS SHOWN HERE
{"message":"View the Blockchain\n{\"ds_chain\" : [{\"index\":0,\"timestamp\":\"Mar 16, 2025, 8:19:38 PM\",\"data\":\"Genesis\",
 \"difficulty\":2,\"nonce\":0,\"previousHash\":\"\"}\n{\"index\":1,\"timestamp\":\"Mar 16, 2025, 8:20:30 PM\",\"data\":\"Alice pays Bob
 100 DSCoin\",\"difficulty\":4,\"nonce\":191044,\"previousHash\":\"00263e9afb48768ba6a36c519bd597a34719ecef2b328ff1e90d88a70e91cfd5\"}\n
 {\"index\":2,\"timestamp\":\"Mar 16, 2025, 8:20:47 PM\",\"data\":\"Bob pays Tony 30 DSCoin\",\"difficulty\":4,\"nonce\":36236,
 \"previousHash\":\"00000ce74253cdc42889c5f0c8c95f28ac21f7e5a7e47f6d2c0ee9bdebbb4721\"}\n{\"index\":3,\"timestamp\":\"Mar 16, 2025,
 8:21:00 PM\",\"data\":\"Carol pays Donna 10 DSCoin\",\"difficulty\":4,\"nonce\":133395,
 \"previousHash\":\"00009e8a83c2fe71079659022cc35955668fec6943b26070a838c0c3189d595a\"}\n],
 \"chainHash\":\"00009dce700b680da170414278b66f57b2a374377bec904c6b771a3733196ed5\"}"}
Number of Blocks on Chain == 4
Client disconnected.
Waiting for a client...
```

Task 1 Client Source Code in the file ClientTCP.java.

```
import com.google.gson.Gson;

import java.net.*;
import java.io.*;

public class ClientTCP {

   public static void main(String args[]) {
      // arguments supply hostname

      Socket clientSocket = null;
      try {
         // Define the server port to connect to
         int serverPort = 7777;

         // Establish a connection to the server at localhost on the specified port
         clientSocket = new Socket("localhost", serverPort);

         // Input and output streams for communication with the server
         BufferedReader in = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
         PrintWriter out = new PrintWriter(new BufferedWriter(new
OutputStreamWriter(clientSocket.getOutputStream())));

         RequestMessage requestMessage = null; // Request message to be sent to the server
         ResponseMessage responseMessage = null; // Response message received from the
server
         String requestJson = ""; // JSON representation of the request message
```

```
// BufferedReader for reading user input from the console
BufferedReader typed = new BufferedReader(new InputStreamReader(System.in));

// Main loop to interact with the user and send commands to the server
while (true) {
   // Display the menu options to the user
   System.out.println("""
      0. View basic blockchain status.
      1. Add a transaction to the blockchain.
      2. Verify the blockchain.
      3. View the blockchain.
      4. Corrupt the chain.
      5. Hide the corruption by repairing the chain.
      6. Exit""");
   System.out.println("Enter choice:");

   // Read the user's choice
   String choice = typed.readLine();
   String command = ""; // Command data for specific requests

   switch (choice) {
      // Handle cases that require no additional user input
      case "0", "2", "3", "5":
         // Create a request message for the selected option
         requestMessage = new RequestMessage(choice, command);
         requestJson = requestMessage.toJson(); // Convert the request to JSON
         out.println(requestJson); // Send the request to the server
         out.flush(); // Ensure data is sent immediately

         // Read and parse the response message from the server
         responseMessage = ResponseMessage.fromJson(in.readLine());
         System.out.println(responseMessage.toString()); // Display the server's response
         break;

      // Handle adding a transaction to the blockchain
      case "1":
         System.out.println("Enter difficulty > 1"); // Prompt for difficulty level
         String difficulty = typed.readLine();
```

```
                System.out.println("Enter a transaction"); // Prompt for transaction details
                String transaction = typed.readLine();
                command = difficulty + "," + transaction; // Combine inputs into a single command
                requestMessage = new RequestMessage(choice, command); // Create the request
                requestJson = requestMessage.toJson(); // Convert to JSON
                out.println(requestJson); // Send to server
                out.flush(); // Ensure data is sent

                responseMessage = ResponseMessage.fromJson(in.readLine()); // Read server
response
                System.out.println(responseMessage.toString()); // Display the response
                break;

            // Handle corrupting the blockchain
            case "4":
                System.out.println("Corrupt the chain");
                System.out.println("Enter block ID of block to corrupt"); // Prompt for block ID
                String id = typed.readLine();
                System.out.println("Enter new data for block " + id); // Prompt for new data
                String data = typed.readLine();
                command = id + "," + data; // Combine inputs into a single command
                requestMessage = new RequestMessage(choice, command); // Create the request
                requestJson = requestMessage.toJson(); // Convert to JSON
                out.println(requestJson); // Send to server
                out.flush(); // Ensure data is sent

                responseMessage = ResponseMessage.fromJson(in.readLine()); // Read server
response
                System.out.println(responseMessage.toString()); // Display the response
                break;

            // Exit the program
            case "6":
                return;

            // Handle invalid input
            default:
                System.out.println("Invalid choice");
```

```
                    break;
                }
            }
        } catch (IOException e) {
            // Handle exceptions related to input/output operations
            System.out.println("IO Exception:" + e.getMessage());
        } finally {
            try {
                // Close the client socket to release resources
                if (clientSocket != null) {
                    clientSocket.close();
                }
            } catch (IOException e) {
                // Ignore exceptions that occur during socket closure
            }
        }
    }
}

/**
 * Represents a request message with a choice and command.
 * This class is used to communicate between the client and server by converting
 * objects to JSON and vice versa.
 */
class RequestMessage {
    private String choice;  // The choice made by the user (menu option)
    private String command; // The command or data associated with the choice

    /**
     * Constructs a new RequestMessage with the specified choice and command.
     *
     * @param choice  The choice made by the user.
     * @param command The command or data associated with the choice.
     */
    public RequestMessage(String choice, String command) {
        this.choice = choice;
        this.command = command;
    }
```

```
/**
 * Gets the choice.
 *
 * @return The choice made by the user.
 */
public String getChoice() {
   return choice;
}

/**
 * Gets the command.
 *
 * @return The command or data associated with the choice.
 */
public String getCommand() {
   return command;
}

/**
 * Sets the choice.
 *
 * @param choice The choice to set.
 */
public void setChoice(String choice) {
   this.choice = choice;
}

/**
 * Sets the command.
 *
 * @param command The command or data to set.
 */
public void setCommand(String command) {
   this.command = command;
}

/**
```

```
    * Converts this RequestMessage object to a JSON string.
    *
    * @return A JSON representation of this object.
    */
   public String toJson() {
      return new Gson().toJson(this);
   }

   /**
    * Creates a RequestMessage object from a JSON string.
    *
    * @param json The JSON string representing a RequestMessage.
    * @return A RequestMessage object parsed from the JSON string.
    */
   public static RequestMessage fromJson(String json) {
      return new Gson().fromJson(json, RequestMessage.class);
   }
}

/**
 * Represents a response message with a message string.
 * This class is used to communicate responses between the server and client
 * by converting objects to JSON and vice versa.
 */
class ResponseMessage {
   private String message; // The response message content

   /**
    * Constructs a new ResponseMessage with the specified message.
    *
    * @param message The response message to set.
    */
   public ResponseMessage(String message) {
      this.message = message;
   }

   /**
    * Gets the response message.
```

```
     *
     * @return The response message.
     */
    public String getMessage() {
        return message;
    }

    /**
     * Sets the response message.
     *
     * @param message The response message to set.
     */
    public void setMessage(String message) {
        this.message = message;
    }

    /**
     * Returns the string representation of the response message.
     *
     * @return The response message as a string.
     */
    @Override
    public String toString() {
        return message;
    }

    /**
     * Converts this ResponseMessage object to a JSON string.
     *
     * @return A JSON representation of this object.
     */
    public String toJson() {
        return new Gson().toJson(this);
    }

    /**
     * Creates a ResponseMessage object from a JSON string.
     *
```

```
    * @param json The JSON string representing a ResponseMessage.
    * @return A ResponseMessage object parsed from the JSON string.
    */
   public static ResponseMessage fromJson(String json) {
      return new Gson().fromJson(json, ResponseMessage.class);
   }
}
```

Task 1 Server Source Code in the file ServerTCP.java.

```
import java.net.*;
import java.io.*;
import java.util.ArrayList;
import java.util.Scanner;

public class ServerTCP {

   public static void main(String[] args) {
      int serverPort = 7777; // The port number the server will listen on
      // Initialize a shared blockchain instance with an empty list of blocks
      BlockChain blockChain = new BlockChain(new ArrayList<Block>(), "", 0);

      try (ServerSocket listenSocket = new ServerSocket(serverPort)) {
         System.out.println("Server is running...");

         while (true) {
            System.out.println("Waiting for a client...");
            try (Socket clientSocket = listenSocket.accept()) { // Accept incoming client connections
               System.out.println("New client connected.");

               // Set up "in" to read client input
               Scanner in = new Scanner(clientSocket.getInputStream());
               // Set up "out" to send responses to the client
               PrintWriter out = new PrintWriter(new BufferedWriter(new
OutputStreamWriter(clientSocket.getOutputStream())), true);

               // Delegate client interaction to the handleClient method
               handleClient(in, out, blockChain);
            } catch (IOException e) {
```

```
            // Handle client-specific connection errors
            System.out.println("Client connection error: " + e.getMessage());
        }
      }
    } catch (IOException e) {
      // Handle errors related to starting or running the server
      System.out.println("Server error: " + e.getMessage());
    }
}

/**
 * Handles interaction with a single client.
 *
 * @param in        Scanner to read client input.
 * @param out       PrintWriter to send responses to the client.
 * @param blockChain Shared instance of the blockchain.
 */
private static void handleClient(Scanner in, PrintWriter out, BlockChain blockChain) {
    try {
      // Continuously read input from the client
      while (in.hasNextLine()) {
        System.out.println("We have a visitor"); // Indicate a client request is being processed

        // Read the JSON request message from the client
        String data = in.nextLine();
        System.out.println("THE JSON REQUEST MESSAGE IS SHOWN HERE\n" + data);

        // Parse the client's JSON request into a RequestMessage object
        RequestMessage requestMessage = RequestMessage.fromJson(data);

        // Process the request and generate a response
        String response = blockChain.response(requestMessage, blockChain);

        // Wrap the response in a ResponseMessage object and convert it to JSON
        ResponseMessage responseMessage = new ResponseMessage(response);
        String responseJson = responseMessage.toJson();

        // Display the JSON response being sent to the client
```

```
            System.out.println("THE JSON RESPONSE MESSAGE IS SHOWN HERE\n" +
responseJson);

            // Send the JSON response back to the client
            out.println(responseJson);

            // Log the number of blocks in the blockchain for debugging
            System.out.println("Number of Blocks on Chain == " + blockChain.getChainSize());
          }
        } catch (Exception e) {
          // Handle exceptions that occur during request processing
          System.out.println("Error processing client request: " + e.getMessage());
        } finally {
          // Log when a client disconnects
          System.out.println("Client disconnected.");
        }
    }
}



/**
 * Represents a request message with a choice and command.
 * This class is used to communicate between the client and server by converting
 * objects to JSON and vice versa.
 */
class RequestMessage {
    private String choice;  // The choice made by the user (menu option)
    private String command; // The command or data associated with the choice

    /**
     * Constructs a new RequestMessage with the specified choice and command.
     *
     * @param choice  The choice made by the user.
     * @param command The command or data associated with the choice.
     */
    public RequestMessage(String choice, String command) {
        this.choice = choice;
```

```
    this.command = command;
}

/**
 * Gets the choice.
 *
 * @return The choice made by the user.
 */
public String getChoice() {
    return choice;
}

/**
 * Gets the command.
 *
 * @return The command or data associated with the choice.
 */
public String getCommand() {
    return command;
}

/**
 * Sets the choice.
 *
 * @param choice The choice to set.
 */
public void setChoice(String choice) {
    this.choice = choice;
}

/**
 * Sets the command.
 *
 * @param command The command or data to set.
 */
public void setCommand(String command) {
    this.command = command;
}
```

```
    /**
     * Converts this RequestMessage object to a JSON string.
     *
     * @return A JSON representation of this object.
     */
    public String toJson() {
        return new Gson().toJson(this);
    }

    /**
     * Creates a RequestMessage object from a JSON string.
     *
     * @param json The JSON string representing a RequestMessage.
     * @return A RequestMessage object parsed from the JSON string.
     */
    public static RequestMessage fromJson(String json) {
        return new Gson().fromJson(json, RequestMessage.class);
    }
}



/**
 * Represents a response message with a message string.
 * This class is used to communicate responses between the server and client
 * by converting objects to JSON and vice versa.
 */
class ResponseMessage {
    private String message; // The response message content

    /**
     * Constructs a new ResponseMessage with the specified message.
     *
     * @param message The response message to set.
     */
    public ResponseMessage(String message) {
        this.message = message;
```

```
}

/**
 * Gets the response message.
 *
 * @return The response message.
 */
public String getMessage() {
   return message;
}

/**
 * Sets the response message.
 *
 * @param message The response message to set.
 */
public void setMessage(String message) {
   this.message = message;
}

/**
 * Returns the string representation of the response message.
 *
 * @return The response message as a string.
 */
@Override
public String toString() {
   return message;
}

/**
 * Converts this ResponseMessage object to a JSON string.
 *
 * @return A JSON representation of this object.
 */
public String toJson() {
   return new Gson().toJson(this);
}
```

```
    /**
     * Creates a ResponseMessage object from a JSON string.
     *
     * @param json The JSON string representing a ResponseMessage.
     * @return A ResponseMessage object parsed from the JSON string.
     */
    public static ResponseMessage fromJson(String json) {
        return new Gson().fromJson(json, ResponseMessage.class);
    }
}
```

Project3Task2SigningClient

```
import com.google.gson.Gson;

import java.math.BigInteger;
import java.net.*;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;

public class SigningClientTCP {

    public static void main(String[] args) {
        // arguments supply hostname

        Socket clientSocket = null;
        try {
            // Define the server port to connect to
            int serverPort = 7777;

            // Establish a connection to the server at localhost on the specified port
            clientSocket = new Socket("localhost", serverPort);

            // Input and output streams for communication with the server
            BufferedReader in = new BufferedReader(new
```

```
InputStreamReader(clientSocket.getInputStream()));
        PrintWriter out = new PrintWriter(new BufferedWriter(new
OutputStreamWriter(clientSocket.getOutputStream())));

        RequestMessage requestMessage = null; // Request message to be sent to the server
        ResponseMessage responseMessage = null; // Response message received from the
server
        String requestJson = ""; // JSON representation of the request message

        // BufferedReader for reading user input from the console
        BufferedReader typed = new BufferedReader(new InputStreamReader(System.in));

        // Main loop to interact with the user and send commands to the server
        while (true) {
          // Display the menu options to the user
          System.out.println("""
            0. View basic blockchain status.
            1. Add a transaction to the blockchain.
            2. Verify the blockchain.
            3. View the blockchain.
            4. Corrupt the chain.
            5. Hide the corruption by repairing the chain.
            6. Exit""");
          System.out.println("Enter choice:");

          // Read the user's choice
          String choice = typed.readLine();
          String command = ""; // Command data for specific requests

          RSA rsa = new RSA();
          SigningClientTCP signingClientTCP = new SigningClientTCP();
          String ID = signingClientTCP.calculateID(rsa.getE(),rsa.calculateN());
          MessageSign messageSign = new MessageSign(rsa.calculateD(),rsa.calculateN());
          System.out.println("Public Key: "+rsa.getE().add(rsa.calculateN()));
          System.out.println("Private Key: "+rsa.calculateD().add(rsa.calculateN()));
          String hash = "";
          String signature = "";
```

```
            switch (choice) {
                // Handle cases that require no additional user input
                case "0", "2", "3", "5":
                    hash = ID+(rsa.getE().add(rsa.calculateN())).toString()+choice+command;
                    signature = messageSign.sign(signingClientTCP.calculateHash(hash));
                    // Create a request message for the selected option
                    requestMessage = new RequestMessage(choice, command, ID, rsa.getE(),
rsa.calculateN(), signature);
                    requestJson = requestMessage.toJson(); // Convert the request to JSON
                    out.println(requestJson); // Send the request to the server
                    out.flush(); // Ensure data is sent immediately

                    // Read and parse the response message from the server
                    // System.out.println(in.readLine());
                    responseMessage = ResponseMessage.fromJson(in.readLine());
                    System.out.println(responseMessage.toString()); // Display the server's response
                    break;

                // Handle adding a transaction to the blockchain
                case "1":
                    System.out.println("Enter difficulty > 1"); // Prompt for difficulty level
                    String difficulty = typed.readLine();
                    System.out.println("Enter a transaction"); // Prompt for transaction details
                    String transaction = typed.readLine();
                    command = difficulty + "," + transaction; // Combine inputs into a single command

                    hash = ID+(rsa.getE().add(rsa.calculateN())).toString()+choice+command;
                    signature = messageSign.sign(signingClientTCP.calculateHash(hash));

                    requestMessage = new RequestMessage(choice, command, ID, rsa.getE(),
rsa.calculateN(), signature); // Create the request
                    requestJson = requestMessage.toJson(); // Convert to JSON
                    out.println(requestJson); // Send to server
                    out.flush(); // Ensure data is sent

                    // System.out.println(in.readLine());
                    responseMessage = ResponseMessage.fromJson(in.readLine()); // Read server
response
```

```
                System.out.println(responseMessage.toString()); // Display the response
                break;

            // Handle corrupting the blockchain
            case "4":
                System.out.println("Corrupt the chain");
                System.out.println("Enter block ID of block to corrupt"); // Prompt for block ID
                String id = typed.readLine();
                System.out.println("Enter new data for block " + id); // Prompt for new data
                String data = typed.readLine();
                command = id + "," + data; // Combine inputs into a single command

                hash = ID + (rsa.getE().add(rsa.calculateN())).toString() + choice + command;
                signature = messageSign.sign(signingClientTCP.calculateHash(hash));


                requestMessage = new RequestMessage(choice, command, ID, rsa.getE(),
rsa.calculateN(), signature); // Create the request
                requestJson = requestMessage.toJson(); // Convert to JSON
                out.println(requestJson); // Send to server
                out.flush(); // Ensure data is sent

                // System.out.println(in.readLine());
                responseMessage = ResponseMessage.fromJson(in.readLine()); // Read server
response
                System.out.println(responseMessage.toString()); // Display the response
                break;

            // Exit the program
            case "6":
                return;

            // Handle invalid input
            default:
                System.out.println("Invalid choice");
                break;
        }
    }
```

```
    } catch (IOException e) {
        // Handle exceptions related to input/output operations
        System.out.println("IO Exception:" + e.getMessage());
    } catch (Exception e) {
        throw new RuntimeException(e);
    } finally {
        try {
            // Close the client socket to release resources
            if (clientSocket != null) {
                clientSocket.close();
            }
        } catch (IOException e) {
            // Ignore exceptions that occur during socket closure
        }
    }
}
public java.lang.String calculateID(BigInteger e, BigInteger n) {
    try {
        // Create a SHA-256 MessageDigest instance
        MessageDigest digest = MessageDigest.getInstance("SHA-256");

        // Concatenate the block's properties to form the input string
        String input = (e.add(n)).toString();

        // Generate the hash as a byte array
        byte[] hashBytes = digest.digest(input.getBytes(StandardCharsets.UTF_8));

        byte[] last20 = new byte[20];
        System.arraycopy(hashBytes, hashBytes.length-20, last20, 0, 20);

        StringBuilder hexString = new StringBuilder();
        for(byte b: last20){
            hexString.append(String.format("%02x",b));
        }
        return hexString.toString(); // Return the hash as a string
    } catch (Exception exception) {
        throw new RuntimeException(exception); // Handle exceptions
    }
```

```
    }
    public java.lang.String calculateHash(String input) {
        try {
            // Create a SHA-256 MessageDigest instance
            MessageDigest digest = MessageDigest.getInstance("SHA-256");

            // Generate the hash as a byte array
            byte[] hashBytes = digest.digest(input.getBytes(StandardCharsets.UTF_8));

            // Convert the byte array to a hexadecimal string
            StringBuilder hexString = new StringBuilder();
            for (byte b : hashBytes) {
                String hex = Integer.toHexString(0xff & b);
                if (hex.length() == 1) {
                    hexString.append('0'); // Ensure two-character representation
                }
                hexString.append(hex);
            }
            return hexString.toString(); // Return the hash as a string
        } catch (Exception exception) {
            throw new RuntimeException(exception); // Handle exceptions
        }
    }
}
```

Project3Task2VerifyingServer

```
import java.math.BigInteger;
import java.net.*;
import java.io.*;
import java.util.ArrayList;
import java.util.Scanner;
import java.util.concurrent.Executors;

public class VerifyingServerTCP {

    public static void main(String[] args) {
```

```
        int serverPort = 7777; // Server port
        BlockChain blockChain = new BlockChain(new ArrayList<>(), "", 0); // Shared blockchain
instance

        try (ServerSocket listenSocket = new ServerSocket(serverPort)) {
            System.out.println("Server is running...");
            var threadPool = Executors.newCachedThreadPool(); // Thread pool for handling clients

            while (true) {
                System.out.println("Waiting for a client...");
                Socket clientSocket = listenSocket.accept();
                System.out.println("New client connected.");
                threadPool.execute(() -> handleClient(clientSocket, blockChain));
            }
        } catch (IOException e) {
            System.err.println("Server error: " + e.getMessage());
            e.printStackTrace();
        }
    }

    private static void handleClient(Socket clientSocket, BlockChain blockChain) {
        try (Scanner in = new Scanner(clientSocket.getInputStream());
             PrintWriter out = new PrintWriter(new BufferedWriter(new
OutputStreamWriter(clientSocket.getOutputStream())), true)) {

            while (in.hasNextLine()) {
                String data = in.nextLine();
                System.out.println("THE JSON REQUEST MESSAGE IS SHOWN HERE\n" + data);
                String responseJson;
                try {
                    // Parse the request
                    RequestMessage requestMessage = RequestMessage.fromJson(data);

                    // Verify ID and Signature
                    SigningClientTCP signingClientTCP = new SigningClientTCP();
                    MessageVerify messageVerify = new MessageVerify(requestMessage.getE(),
requestMessage.getN());
                    System.out.println("Client's public key:
```

```
"+requestMessage.getE().add(requestMessage.getN()));
                // Calculate hash for verification
                String concatHash =
requestMessage.getID()+(requestMessage.getE().add(requestMessage.getN())).toString()+reque
stMessage.getChoice()+requestMessage.getCommand();
                String hash = signingClientTCP.calculateHash(concatHash);

                // Check validity of the request
                if (!signingClientTCP.calculateID(requestMessage.getE(),
requestMessage.getN()).equals(requestMessage.getID())) {
                    String errorResponse = new ResponseMessage("Invalid ID").toJson();
                    System.out.println("Invalid ID.");
                    out.println(errorResponse);
                    return;
                }

                if (!messageVerify.verify(hash, requestMessage.getSignature())) {
                    String errorResponse = new ResponseMessage("Invalid signature").toJson();
                    System.out.println("Invalid signature.");
                    out.println(errorResponse);
                    return;
                }else
                    System.out.println("Signature verified.");

                // If valid, process the request
                String response = blockChain.response(requestMessage.getChoice(),
requestMessage.getCommand(), blockChain);
                ResponseMessage responseMessage = new ResponseMessage(response);
                responseJson = responseMessage.toJson();
                out.println(responseJson);

            } catch (Exception e) {
                responseJson = new ResponseMessage("Error processing request: " +
e.getMessage()).toJson();
                e.printStackTrace();
            }

            // Send response
```

```
            System.out.println("THE JSON RESPONSE MESSAGE IS SHOWN HERE:\n" +
responseJson);
            out.println(responseJson);
            System.out.println("Number of Blocks on Chain: " + blockChain.getChainSize());
          }
        } catch (IOException e) {
          System.err.println("Client error: " + e.getMessage());
        } finally {
          try {
            clientSocket.close();
          } catch (IOException e) {
            System.err.println("Error closing client socket: " + e.getMessage());
          }
          System.out.println("Client disconnected.");
        }
    }
}
```

Project3Task2ClientConsole

```
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
0
Public Key:
 596381203661790523983288119738229559199931054047815890347622865335997827394310635140399082807193788665985518041959073473327944431947715
 884543026745345486629183492940026819151703067133070280218885435496466791346219155696212997453880781058150 6
Private Key:
 646012146255613944612930205965128929263809839600660663009721147337653871317721383329295989861688729285548284348028084668849040167605831
 894358705289769365316789979258252686832214637888551365171987192595437002919215853364009271878191696716409 0
{"message":"Current size of chain: 1\nDifficulty of most recent block: 2\nTotal difficulty for all blocks: 2\nExperimented with 1
 hashes\nApproximate hashes per second on this machine: 4866180\nExpected total hashes required for the whole chain: 256.0\nNonce for
 most recent block: 2\nChain hash: 006c95ef843d8fc8dc0c181e9c1b56f9eb63528b55d02a3b431313aec6462a82"}
```

```
Current size of chain: 1
Difficulty of most recent block: 2
Total difficulty for all blocks: 2
Experimented with 1 hashes
Approximate hashes per second on this machine: 4866180
Expected total hashes required for the whole chain: 256.0
Nonce for most recent block: 2
Chain hash: 006c95ef843d8fc8dc0c181e9c1b56f9eb63528b55d02a3b431313aec6462a82
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
1
Public Key:
 5468452911784659981545168317055947251269933781877830679883915428210746699612558941191217931907073812204468466319566683171619778282056457495217186098534438595718478409225882192425252835956018676596459412838973484736806656337810034828897711706
Private Key:
```

```
Private Key:
 9436058019668154528480071554555532093387903040896260709157153120070627462640728946739876135092775931165093389240899606323237408496132608672801200680385247390051376481394031110285899239283636763122346888442884247523685278018861639135568305042
Enter difficulty > 1
2
Enter a transaction
assd
{"message":"Total execution time to add this block was 21 milliseconds"}
Total execution time to add this block was 21 milliseconds
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
3
Public Key:
 5396555061604152535322850857045054989151724365269108181818841259895229426535317556942106206489794845091912608305595370020300415594387978994094512291831128348684284384963584883009507779892011472572371630265363402146755721480015865648690010638
Private Key:
```

```
Private Key:
 8595193714090737292553364025818650989176118722180143281422925803597720951532867193016630758896772289247051617867077466366762216662023498961513969825853935498119055868803612789692749193164701770121324633121889727171596739589999677178692716314
{"message":"View the Blockchain\n{\"ds_chain\" : [{\"index\":0,\"timestamp\":\"Mar 16, 2025, 7:43:15 PM\",\"data\":\"Genesis\",
 \"difficulty\":2,\"nonce\":2,\"previousHash\":\"\"}\n{\"index\":1,\"timestamp\":\"Mar 16, 2025, 7:43:31 PM\",\"data\":\"assd\",
 \"difficulty\":2,\"nonce\":268,\"previousHash\":\"006c95ef843d8fc8dc0c181e9c1b56f9eb63528b55d02a3b431313aec6462a82\"}\n],
 \"chainHash\":\"00d35c1dc87c988708b2b10e0b185f16b9d84e4546b829d062b53486decffdf7\"}"}
View the Blockchain
{"ds_chain" : [{"index":0,"timestamp":"Mar 16, 2025, 7:43:15 PM","data":"Genesis","difficulty":2,"nonce":2,"previousHash":""}
{"index":1,"timestamp":"Mar 16, 2025, 7:43:31 PM","data":"assd","difficulty":2,"nonce":268,
 "previousHash":"006c95ef843d8fc8dc0c181e9c1b56f9eb63528b55d02a3b431313aec6462a82"}
], "chainHash":"00d35c1dc87c988708b2b10e0b185f16b9d84e4546b829d062b53486decffdf7"}
0. View basic blockchain status.
1. Add a transaction to the blockchain.
2. Verify the blockchain.
3. View the blockchain.
4. Corrupt the chain.
5. Hide the corruption by repairing the chain.
6. Exit
Enter choice:
6
```

```
Enter choice:
6
Public Key:
 353440705262920886652380084877802070591142834974899296105885331143284718771842346662633668449583505708500401841250702676461985670586524
 327695779178527516027613741615435615379403701723717357736029546797508725873797451417932858404389281459838 0
Private Key:
 458318276638602142778367473843098267652130593214624437802501777345330249173844545657830838755503069245330090636996654046555619554587640
 244656028101949785714765018972395165429056146883060475366062986291251710295864294627974829381641375111067 6

Process finished with exit code 0
```

Project3Task2ServerConsole

```
Server is running...
Waiting for a client...
New client connected.
Waiting for a client...
THE JSON REQUEST MESSAGE IS SHOWN HERE
{"choice":"0","command":"","ID":"f01f9b52f5150c9f381d974c01fcb1761abb75c2","e":65537,
 "n
 ":59638120366179052398328811973822955919993105404781589034762286533599782739431063514039908280719378866598551804195907347332794443194771
 588454302674534548662918349294002681915170306713307028021888543549646679134621915569621299745388078105159 69,
 "signature
 ":"50028839904703827824049961089292747963155825665858237650301150063654966384640263411692553345104392665338098203945648854063833341973 9
 866815366495671467980611431682367455986567009866416020949726056782038441618081936317045567791508380139115826 4"}
Client's public key:
 59638120366179052398328811973822955919993105404781589034762286533599782739431063514039908280719378866598551804195907347332794443194771 5
 884543026745345486629183492940026819151703067133070280218885435496466791346219155696212997453880781058150 6
Signature verified.
THE JSON RESPONSE MESSAGE IS SHOWN HERE:
```

```
THE JSON RESPONSE MESSAGE IS SHOWN HERE:
{"message":"Current size of chain: 1\nDifficulty of most recent block: 2\nTotal difficulty for all blocks: 2\nExperimented with 1
 hashes\nApproximate hashes per second on this machine: 4866180\nExpected total hashes required for the whole chain: 256.0\nNonce for
 most recent block: 2\nChain hash: 006c95ef843d8fc8dc0c181e9c1b56f9eb63528b55d02a3b431313aec6462a82"}
Number of Blocks on Chain: 1
THE JSON REQUEST MESSAGE IS SHOWN HERE
{"choice":"1","command":"2,assd","ID":"e7dcd0ef923ca363f907de749a66c342608f1517","e":65537,
 "n
 ":54684529117846599815451683170559472512699337818778306798839154282107466996125589411912179319070738122044684663195666831716197782820564
 574952171860985344385957184784092258821924252528359560186765964594128389734847368066563378100348288976461 69,
 "signature
 ":"28244675398353986378810639856284499825527010964202928389100613365270193856854429452614312123187568092951816240327666178166677778527 0
 647923562867736444761779899184382463654494908099833066238251095059591235916357078044559991393613812495294892 3"}
Client's public key:
 54684529117846599815451683170559472512699337818778306798839154282107466996125589411912179319070738122044684663195666831716197782820564 5
 749521718609853443859571847840922588219242525283595601867659645941283897348473680665633781003482889771170 6
Signature verified.
THE JSON RESPONSE MESSAGE IS SHOWN HERE:
{"message":"Total execution time to add this block was 21 milliseconds"}
Number of Blocks on Chain: 2
```

```
Number of Blocks on Chain: 2
THE JSON REQUEST MESSAGE IS SHOWN HERE
{"choice":"3","command":"","ID":"14f1cb8d24b4ea680618723600fc1e42a98cb868","e":65537,
 "n
 ":539655506160415253532285085704505498915172436526910818181884125989522942653531755694210620648979484509191260830559537002030041559438797899409451229183112834868428438496358488300950777989201147257237163026536340214675572148001586564868994510 1,
 "signature
 ":"377788354590336187832068706245222214058434346052930265123325577959585983235907199070136332369615670129184258336572372551943626744766822669904333673524171126718406261334085270985859450731912078684159450043230018331896319478866686276356966075"}
Client's public key:
 539655506160415253532285085704505498915172436526910818181884125989522942653531755694210620648979484509191260830559537002030041559438797899409451229183112834868428438496358488300950777989201147257237163026536340214675572148001586564869001 0638
Signature verified.
THE JSON RESPONSE MESSAGE IS SHOWN HERE:
{"message":"View the Blockchain\n{\"ds_chain\" : [{\"index\":0,\"timestamp\":\"Mar 16, 2025, 7:43:15 PM\",\"data\":\"Genesis\",
 \"difficulty\":2,\"nonce\":2,\"previousHash\":\"\"}\n{\"index\":1,\"timestamp\":\"Mar 16, 2025, 7:43:31 PM\",\"data\":\"assd\",
 \"difficulty\":2,\"nonce\":268,\"previousHash\":\"006c95ef843d8fc8dc0c181e9c1b56f9eb63528b55d02a3b431313aec6462a82\"}\n],
 \"chainHash\":\"00d35c1dc87c988708b2b10e0b185f16b9d84e4546b829d062b53486decffdf7\"}"}
Number of Blocks on Chain: 2
Client disconnected.
```